第二章 线性表

- 2.1 线性表的类型定义
- 2.2 线性表的顺序表示和实现
- 2.3 线性表的链式表示和实现
  - 2.3.1 线性链表
  - 2.3.2 循环链表
  - 2.3.3 双向链表
- 2.4 一元多项式的表示及相加

2.1 线性表的类型定义

例1 26个英文字母组成的字母表

（A，B，C，…，Z）

例2 中国男篮队员情况登记表如下：

姓名	号码	生日	身高	位置
陈江华	4	1989	1.85	后卫
……	……	……	……	……
易建联	11	1987	2.12	大前锋/中锋
姚　明	13	1980	2.26	中锋
王治郅	14	1977	2.16	大前锋/中锋
……	……	……	……	……

线性表的定义

- **线性表：** 由n(n≧0)个数据元素(结点)a_1，a_2，…a_n组成的有限序列, 其中数据元素的个数n定义为表的长度。当n=0时称为空表，常常将非空的线性表(n>0)记作：

  (a_1，a_2，…a_n)

  这里的数据元素a_i (1≦i≦n)只是一个抽象的符号，其具体含义在不同的情况下可以不同。

线性表的逻辑特征

- 对非空的线性表，
  - 有且仅有一个开始结点a_1，它没有直接前趋；
  - 有且仅有一个终端结点a_n，它没有直接后继；
  - 其余的内部结点$a_i(2 \leqq i \leqq n-1, n>3)$都有且仅有一个直接前趋$a_{i-1}$和一个直接后继$a_{i+1}$。

- 线性表是一种典型的线性结构。

抽象数据类型线性表

ADT List{

数据对象：$D=\{a_i \mid a_i \in ElemSet, i=1,2,\ldots,n, n\geq 0\}$

数据关系：$R1=\{< a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, i=2,3,\ldots,n\}$

数据操作：

InitList(&L);

DestroyList(&L);

ClearList(&L);

ListEmpty(L);

ListLength(L);

GetElem(L, i, &e);

LocateElem(L, e, compare( ));

ListInsert(&L, i, e);

ListDelete(&L, i, &e);

……

}ADT List

线性表的合并

例2-1 假设两个线性表LA和LB分别表示两个集合A和B，求新的集合A∪B

分析：

将所有在线性表LB中但不在LA中的元素插入到LA中。

操作步骤：

1．从线性表 LB 中依次察看每个数据元素;

GetElem(LB, i)→e

2．依值在线性表 LA 中进行查访;

LocateElem(LA, e, equal( ))

3．若不存在，则插入之。

ListInsert(LA, n+1, e)

(n 表示线性表 LA 当前长度)

线性表的合并算法

```
void   union(List &La, List Lb){
         La_len=ListLength(La); // 求线
         Lb_len=ListLength(Lb);
         for( i=1; i<=Lb_len; i++){
                  GetElem(Lb, i, &e) ; // 取Lb中第i个数据元素赋给e
                  if(!LocateElem(La, e, equal))
                           ListInsert(La, ++La_len, e);
          // La中不存在和 e 相同的数据元素，则插
         }
}
```

这里 equal是数据元素判定函数

先增 1，然后再传递给函数

假定GetElem与ListInsert执行时间与表长无关，LocateElem执行时间与表长成正比，则该算法的时间复杂度为 O(ListLength(La)* ListLength(Lb))

有序线性表的合并

例2-2 假设两个线性表LA和LB的数据元素按值非递减有序排列，将它们合并成一个新表LC，其元素仍然按值非递减有序排列。

分析：

书P20-21

有序线性表的合并算法

```
void MergeList(List La, List Lb, List &Lc){
        InitList(Lc); i=j=1; k=0;
        La_len=ListLength(La);  Lb_len= ListLength(Lb);
        while((i<= La.len)&&(j<=Lb.len)){
                GetElem(La, i, &ai); GetElem(Lb, j, &bj);
                if(ai<=bj){
                        ListInsert(Lc, ++k, ai); ++i;}
                else{
                        ListInsert(Lc, ++k, bj); ++j;}
        }
        while(i<=La.len){
                GetElem(La, i++, &ai); ListInsert(Lc, ++k, ai);}
        while(j<=Lb.len){
                GetElem(Lb, j++, &bj); ListInsert(Lc, ++k, bj);}}
```

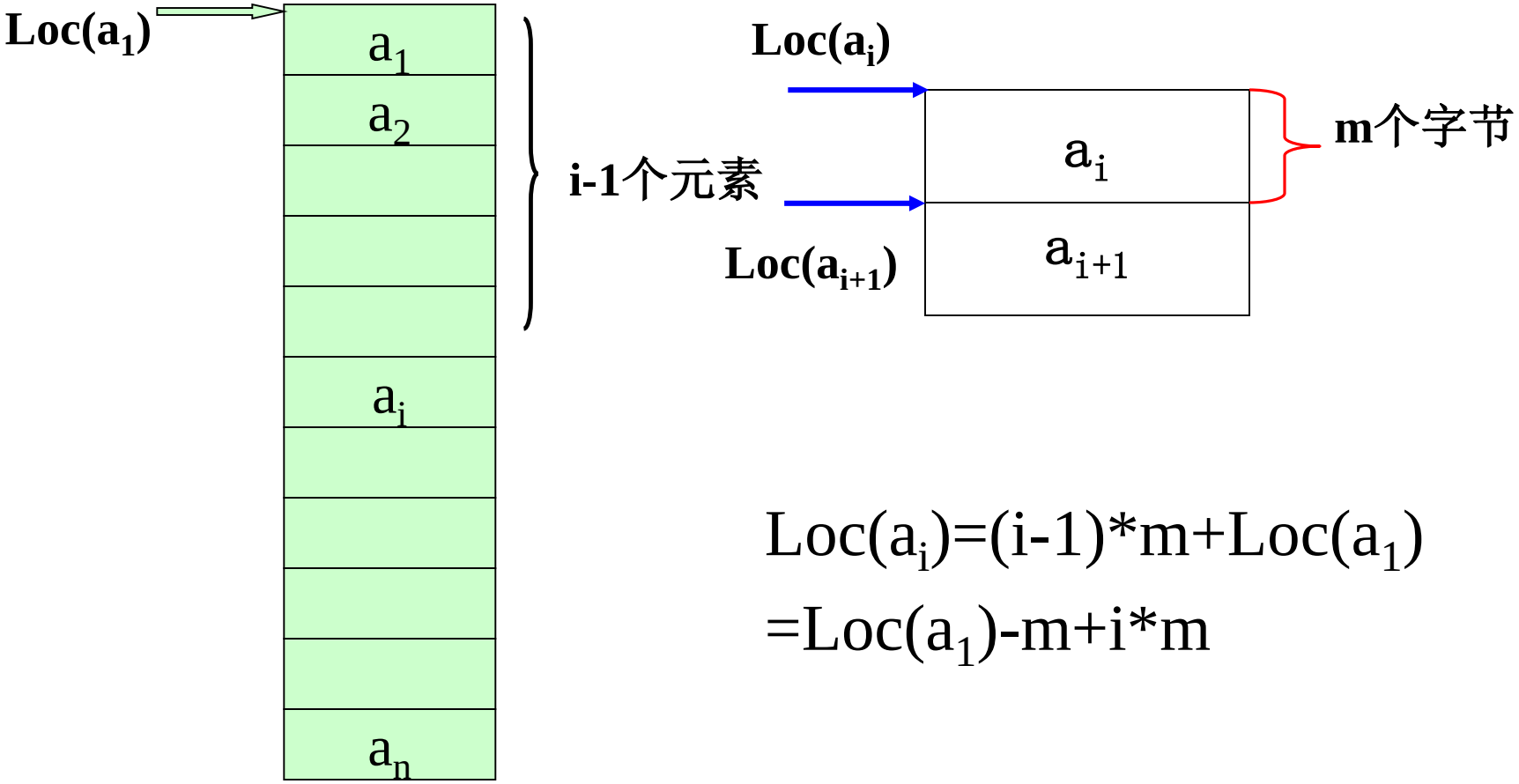
时间复杂度为：O(ListLength(La)+ ListLength(Lb))

2.2 线性表的顺序表示和实现

- **顺序表** 把线性表的结点按逻辑顺序依次存放在一组地址连续的存储单元里。用这种方法存储的线性表简称顺序表。
- 假设线性表的每个元素需占用m个存储单元，并以所占的第一个单元的存储地址作为数据元素的存储位置作为参考点。则线性表中第i+1个数据元素的存储位置$Loc(a_{i+1})$和第i个数据元素的存储位置$Loc(a_i)$之间满足下列关系：

$$Loc(a_{i+1})=Loc(a_i)+m$$

线性表的存储位置

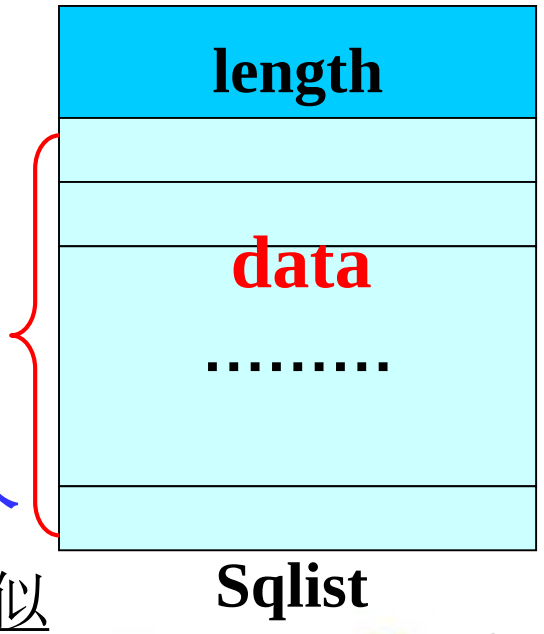


$$Loc(a_i)=(i-1)*m+Loc(a_1)$$
$$=Loc(a_1)-m+i*m$$

用数组类型来描述顺序表

- 由于在高级语言中的一维数组也是采用顺序存储表示，故可以用数组类型来描述顺序表。又因为除了用数组来存储线性表的元素之外，顺序表还应该用一个变量来表示线性表的长度属性，利用C++语言的结构类型来定义顺序表类型如下：

```
# define ListSize   100  //表容量
typedef  int  DataType; //以int为例
struct Sqlist{
      DataType  data[ListSize];
      int   length;//当前表中元素数
};
```



此定义SqList与书上的定义（p22）相似

顺序表上实现的基本操作

- 插入操作

线性表的插入运算是指在表的i(1≦i≦n+1)个位置上，插入一个新结点x，使长度为n的线性表

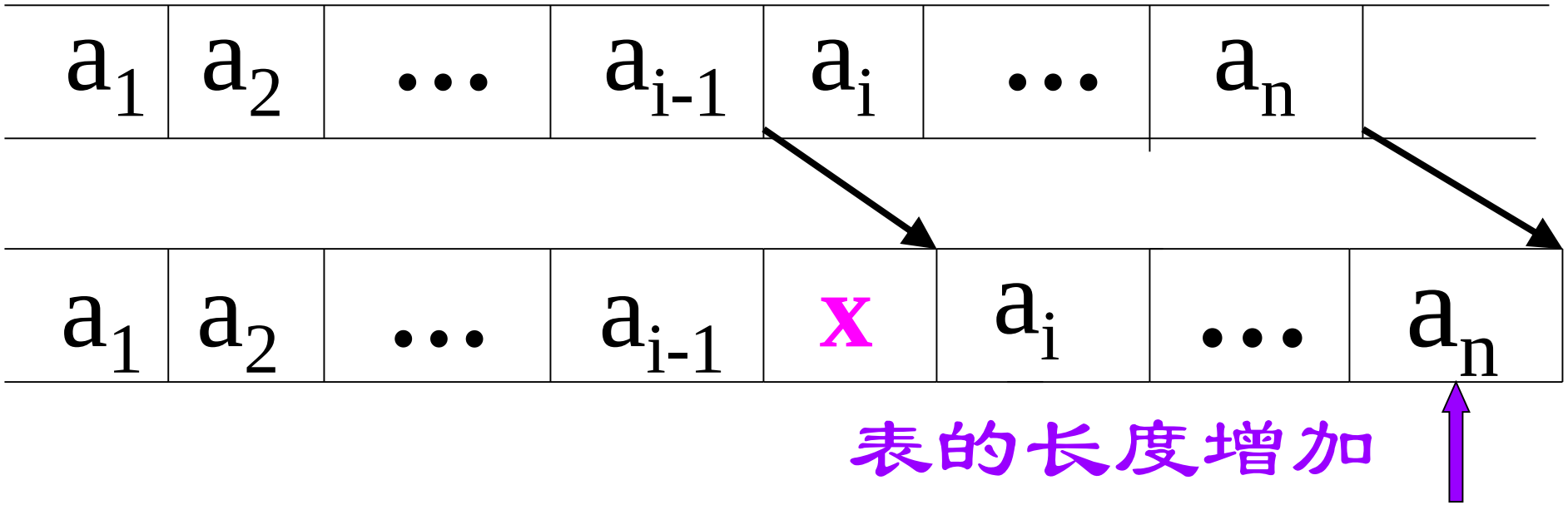
$(a_1, \ldots a_{i-1}, a_i, \ldots, a_n)$

变成长度为n+1的线性表

$(a_1, \ldots a_{i-1}, x, a_i, \ldots, a_n)$

插入操作分析

$<a_{i-1}, a_i>$ ⟶ $<a_{i-1}, x>$, $<x, a_i>$



插入操作算法

```
void InsertList(&L,x,i)
//在线性表L中第i个位置插入元素x
{   if(i<1 || i>L.length+1){
      cout<<“插入序号错误”<<endl;
          return ERROR;}
  if(L.length>=ListSize) 溢出处理;
else  {
    for(j=L.length-1;j>=i-1;j--)
    //第i个元素（下标为i-1)开始
            L.data[j+1]=L.data[j];//顺序后移
    L.data[i-1]=x;
    L.length++;   }
 }
```

注意：

C/C++语言中的数组下标从“0”开始，因此，若L是Sqlist类型的顺序表，则表中第i个元素位置是L.data[i-1]。

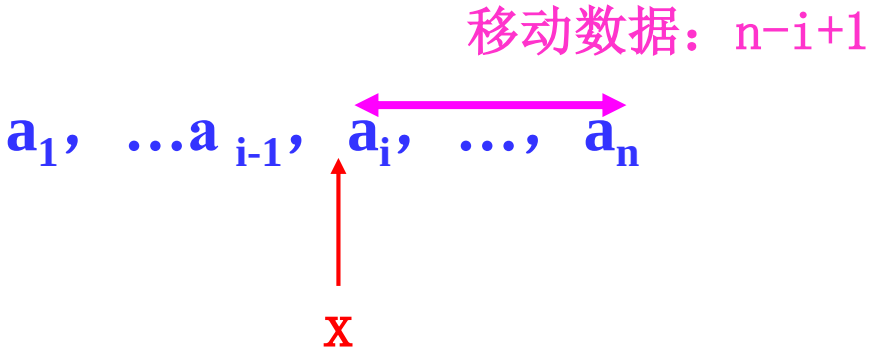
插入操作算法分析

- 设表的长度为n。该算法的时间主要化费在插入结点 i 后移语句上，该语句的执行次数（即移动结点的次数）是n-i+1。由此，所需移动结点的次数依赖于

  (1) 表的长度

  (2) 插入位置。

- 当插入位置=n+1时，不移动->最好情况；
- 当插入位置=1时，需移动表中所有结点->最坏情况



插入操作算法的平均移动次数

- 由于插入可能在表中任何位置上进行，在长度为n的线性表中第i个位置上插入一个结点，则在第i个位置上插入一个结点的移动次数为n-i+1。令$E_{is}(n)$表示移动结点的期望值（即移动的平均次数）， P_i代表在第i个位置插入概率，则

$$E_{is}(n)= p_1 * n+p_2*(n-1)+ \ldots.+ p_n*1+p_{n+1} *0$$

- 若表中任何位置$(1\leq i\leq n+1)$上插入结点的概率是均等的，则

$$p_1=p_2=p_3=\ldots=p_{n+1}=1/(n+1)$$

- 因此，在等概率插入的情况下：

$$E_{is}(n)=1/(n+1)[n+(n-1)+\ldots+1+0]=n/2$$

插入操作算法的时间复杂度

- 在顺序表上做插入运算，平均要移动表上一半结点。当表长 n较大时，算法的效率相当低。虽然$E_{is}(n)$中n的的系数较小，但就数量级而言，它仍然是线性阶的。
- 因此算法的平均时间复杂度为O(n)。

删除操作

- 线性表的删除运算是指将表的第i(1≦i≦n)结点删除，使长度为n的线性表：

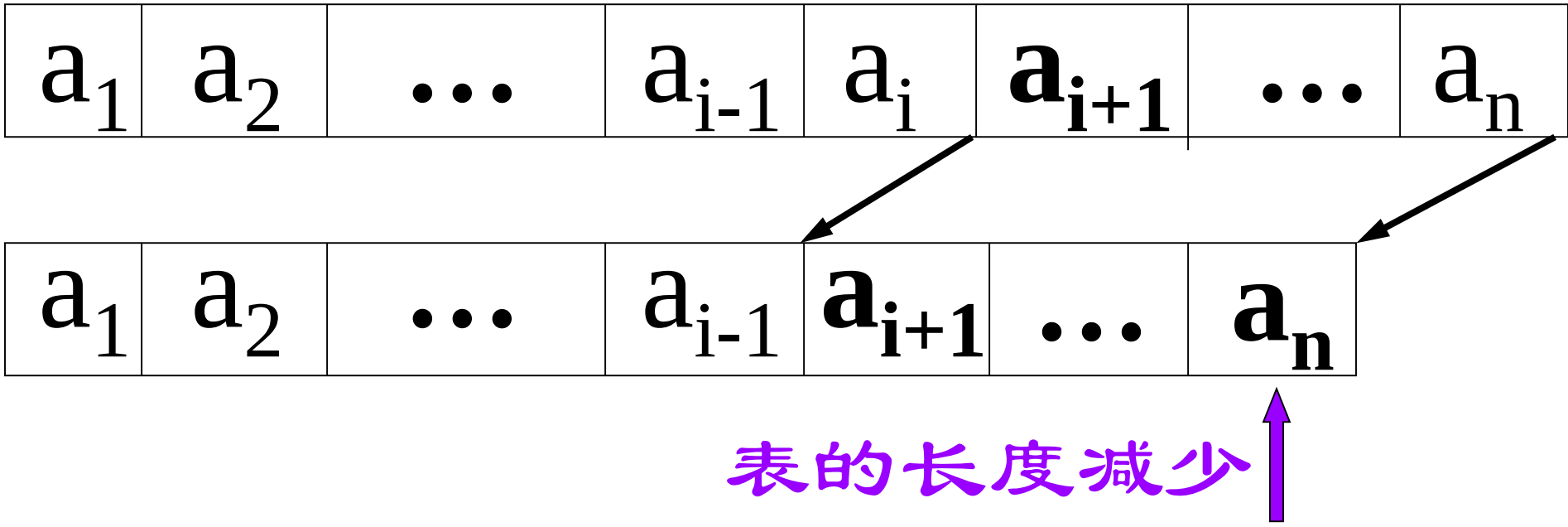
$$(a_1, \ldots a_{i-1}, a_i, a_{i+1} \ldots, a_n)$$

变成长度为n-1的线性表

$$(a_1, \ldots a_{i-1}, a_{i+1}, \ldots, a_n)$$

删除操作分析

$<a_{i-1}, a_i>, <a_i, a_{i+1}>$ ⟹ $<a_{i-1}, a_{i+1}>$



删除操作的算法

```
void deleteList(&L,i)//表L中删除第i个元素
 {
    if(i<1 || i>L.length){
      cout<<“删除序号错”<<endl;
          return ERROR;}
        for(j=i; j<=L.length-1; j++)
             L.data[j-1]=L.data[j];
        L.length--;
   }
```

删除操作算法分析

- 算法分析与插入算法相似，结点的移动次数也是由表长n和位置i决定。

  若i=n，无需移动结点；

  若i=1，则前移元素n-1个

- 令E_{dl}表示所需移动结点的平均次数，删除表中第i个结点的移动次数为n-i， p_i是删除i个元素的概率，则

  $$E_{dl}=p_1*(n-1)+p_2(n-2)+ \ldots.+p_{n-1}*1+p_n*0$$

  等概率的假设下：$p_1=p_2=p_3=\ldots=p_n=1/n$

  $$E_{dl}=1/n[(n-1)+(n-2)+\ldots+1+0]=(n-1)/2$$

- 即在顺序表上做删除运算，平均要移动表中约一半的结点，平均时间复杂度也是O(n)。

思考题：逆序操作

- 试写出在顺序存储结构下逆转线性表的算法，要求使用最小的附加空间。

```
void InvList(&L)
{
        DataType t;
        int n=L.length;
        for(int i=1;  i<=n/2; i++)
        {
                t=L.data[i-1];
                L.data[i-1]= L.data[n-i];
                L.data[n-i]=t;
        }
        return;}
```

```
void InvList(&L)
{
        DataType t;
        int n=L.length;
        for(int i=1;  i<=n/2; i++)
        {
                L.data[i-1]=L.data[i-1]+L.data[n-i];
                L.data[n-i]=L.data[i-1]-L.data[n-i];
                L.data[i-1]=L.data[i-1]-L.data[n-i];
        }
        return;}
```

查找操作

```
//教材上的定义 p22
typedef struct{
        ElemType *elem;
        int length;
        int listsize;
}SqList;
```

```
int LocateElem_Sq(SqList L, ElemType e,
                  Status (*compare) (ElemType, ElemType))
{       //在顺序线性表L中查找第1个值与e满足compare( )的元素的位序
        //若找到，则返回其在L中的位序，否则返回0
        i=1;        //第1个元素的位序
        p=L.elem; //第1个元素的存储位置
        while(i<=L.length && !(*compare)(*p++, e))
                ++i;
        if(i<=L.length)
                return i;
        else
                return 0;
}//教材 P25-26
```

作业2.2

2.2.1 指出以下算法中的错误和低效（即费时）之处，并将它改写成一个既正确又高效的算法。（用C++语言）

```
Status DeleteK(SqList &a, int i, int k) {
    //本过程从顺序存储结构的线性表a中删除第i个元素起的k个元素
    if(i<1 || k<0 || i+k>a.length)
        return INFEASIBLE;
    else{
        for(count=1; count<k; count++) {
            //删除第一个元素
            for(j=a.length; j>=i+1; j--)
                a.elem[j-1]=a.elem[j];
            a.length--;
        }
    return OK;
}//DeleteK
```

```
//教材上的定义 p22
typedef struct{
    ElemType *elem;
    int length;
    int listsize;
}SqList;
```

上机作业2.2

2.2.2 设顺序表L中的数据元素非递减有序。试写一算法，将x插入到顺序表的适当位置上，以保持该表的有序性。

顺序表总结

1. 是线性表的一种表示方法；
2. 是一组地址连续的存储空间；
3. 可用一维数组类型描述，长度可变，数组需动态分配；

优点：可对每个数据元素随机存取，表长是显值；

缺点：在进行插入和删除元素时要进行元素移动，大致移动表长一半的元素。

2.3 线性表的链式表示与实现

- 2.3.1 线性链表
  - 结点和单链表的 C 语言描述
  - 线性表的操作在单链表中的实现
- 2.3.2 循环链表
- 2.3.3 双向链表

2.3.1 线性链表

用一组**地址任意**的存储单元**存放**线性表中的数据元素。链表中元素的逻辑次序和物理次序不一定相同。

以**元素(数据元素的映象)**

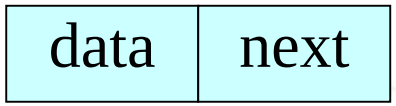
+ 指针(指示后继元素存储位置)

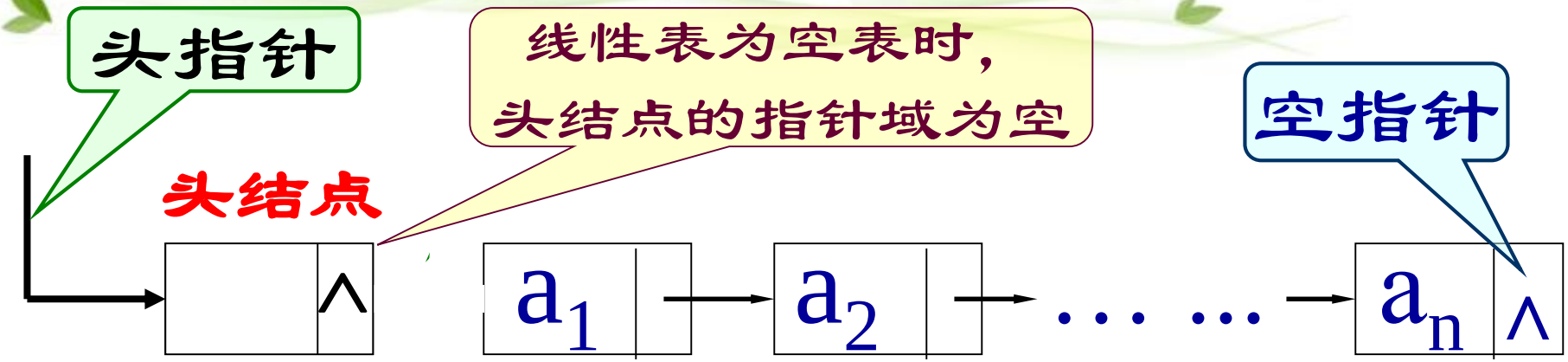
= 结点

(表示数据元素 或 **数据元素的映象**)

以“**结点的序列**”表示线性表

——称作**链表**





以线性表中第一个数据元素 a_1 的存储地址作为线性表的地址，称作线性表的头指针。

有时为了操作方便，在第一个结点之前虚加一个“头结点”，以指向头结点的指针为链表的头指针。

结点和单链表的 C 语言描述

```
Typedef struct  LNode {
   ElemType      data;  // 数据域
   struct Lnode   *next;  // 指针域
} LNode, *LinkList;

LinkList  L;    // L 为单链表的头指针
```

单链表操作的实现

GetElem(L, i, &e)　// 取第i个数据元素

ListInsert(&L, i, e)　// 插入数据元素

ListDelete(&L, i, e)　// 删除数据元素

ClearList(&L)　// 重置线性表为空表

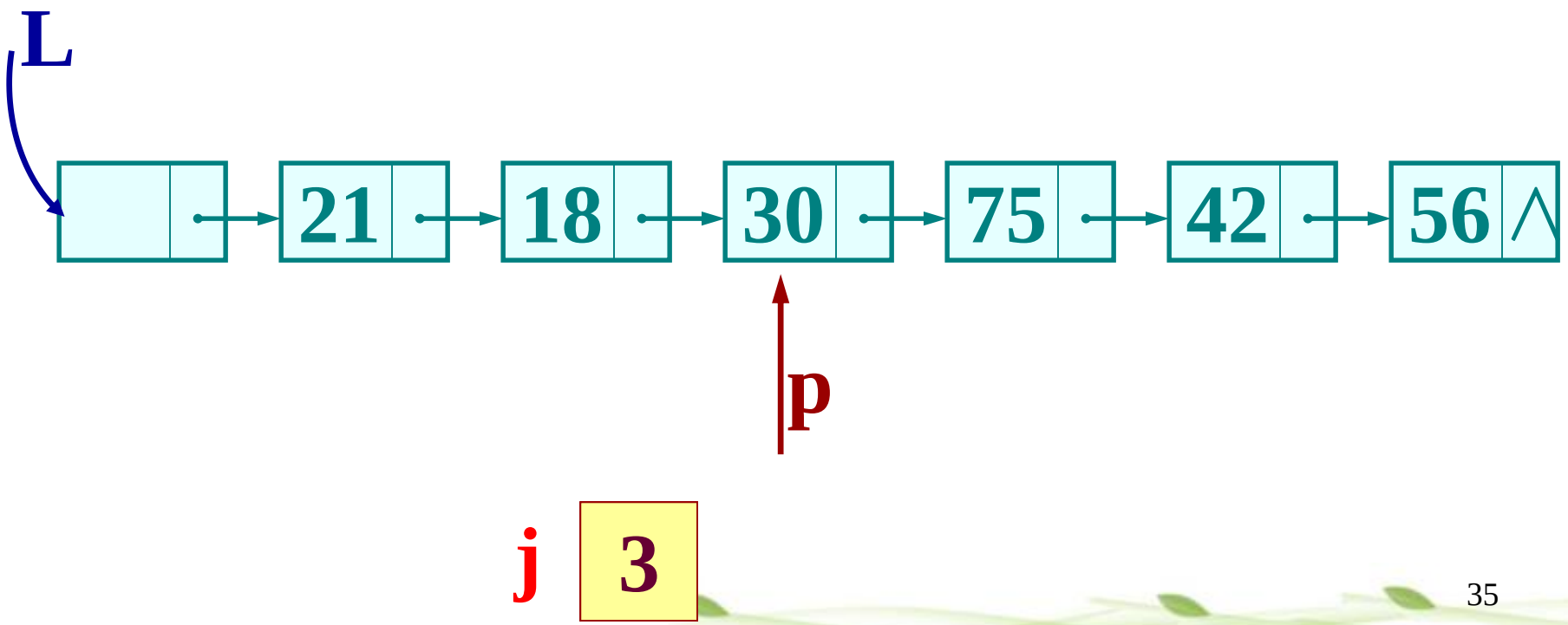
CreateList(&L, n)

// 生成含 *n* 个数据元素的链表

线性表的操作

GetElem(L, i, &e)

在单链表中的实现:



单链表是一种顺序存取的结构，为找第 i 个数据元素，必须先找到第 i-1 个数据元素。

因此，查找第 i 个数据元素的基本操作为：移动指针，比较 j 和 i 。

令指针 p 始终指向线性表中第 j 个数据元素。

```
Status GetElem_L(LinkList L, int i, ElemType &e) {
  // L是带头结点的链表的头指针，以 e 返回第 i 个元素
  p = L->next;   j = 1;  // p指向第一个结点，j为计数器
  while (p && j<i)  { p = p->next;  ++j;  }
        // 顺指针向后查找，直到 p 指向第 i 个元素
      // 或 p 为空
  if ( !p || j>i )
     return ERROR;      // 第 i 个元素不存在
  e = p->data;          // 取得第 i 个元素
  return OK;
} // GetElem_L
```

算法时间复杂度为：

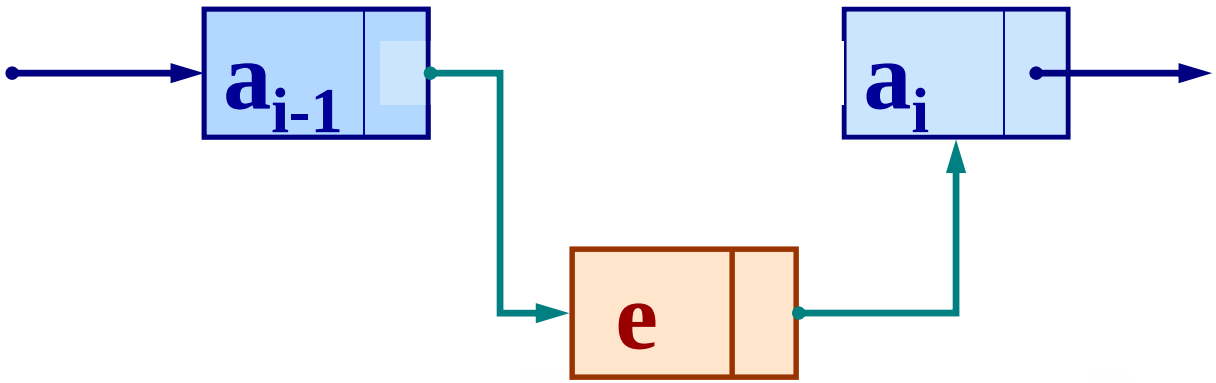
O(ListLength(L))

线性表的操作 ListInsert(&L, i, e)

在单链表中的实现：

有序对 $<a_{i-1}, a_i>$

改变为 $<a_{i-1}, e>$ 和 $<e, a_i>$



可见，在链表中插入结点只需要修改指针。但同时，若要在第 i 个结点之前插入元素，修改的是第 i-1 个结点的指针。

因此，在单链表中第 i 个结点之前进行插入的基本操作为:

找到线性表中第i-1个结点，然后修改其指向后继的指针。

```
Status ListInsert_L(LinkList &L, int i, ElemType e) {
  // L 为带头结点的单链表的头指针，本算法
  // 在链表中第i 个结点之前插入新的元素 e
  p = L;    j = 0;
  while (p && j < i-1)
      { p = p->next;  ++j; }   // 寻找第 i-1 个结点
  if (!p || j > i-1)
       return ERROR;      // i 大于表长或者小于1
      ……
} // LinstInsert_L
```

算法的时间复杂度为： **O(ListLength(L))**

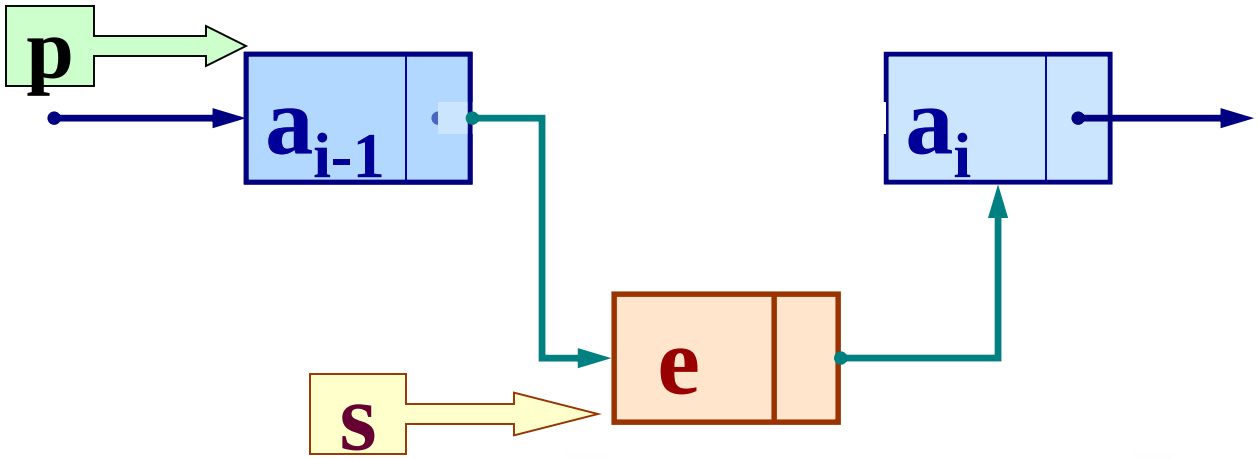
s = (LinkList) **malloc** (**sizeof** (LNode));

// 生成新结点

s->data = e;

s->next = p->next; p->next = s; // 插入

return OK;



线性表的操作**ListDelete (&L, i, &e)**在链表中的实现:

有序对<a_{i-1}, a_i> 和 <a_i, a_{i+1}>

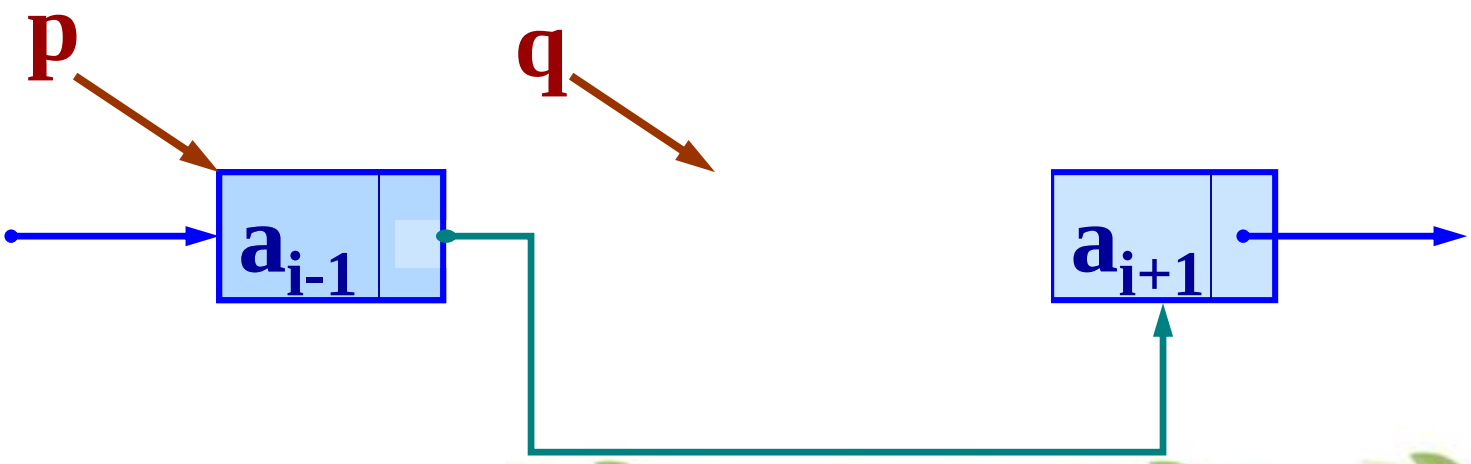
改变为 <a_{i-1}, a_{i+1}>



在单链表中删除第 i 个结点的基本操作为:找到线性表中第i-1个结点，修改其指向后继的指针。

```
q = p->next;   p->next = q->next;

e = q->data;   free(q);
```



```
Status ListDelete_L(LinkList &L, int i, ElemType &e) {
  // 删除以 L 为头指针(带头结点)的单链表中第 i 个结点
    p = L;    j = 0;
    while (p->next && j < i-1) {  p = p->next;   ++j; }
                                  // 寻找第 i 个结点，并令 p 指向其前趋
    if  (!(p->next) || j > i-1)
        return ERROR;  // 删除位置不合理
    q = p->next;   p->next = q->next;  // 删除并释放结点
    e = q->data;   free(q);
    return OK;
} // ListDelete_L
```

算法的时间复杂度为：**O(ListLength(L))**

操作 ClearList(&L) 在链表中的实现:

```
void ClearList(&L) {
  // 将单链表重新置为一个空表
  while (L->next) {
    p=L->next;    L->next=p->next;
    free(p);
  }
} // ClearList
```

算法时间复杂度： **O(ListLength(L))**

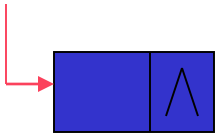
如何从线性表得到单链表？

链表是一个动态的结构，它不需要予分配空间，因此**生成链表的过程**是一个结点“**逐个插入**” 的过程。

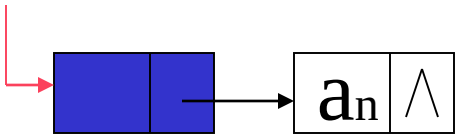
例如：逆位序输入 n 个数据元素的值，建立带头结点的单链表。

操作步骤：

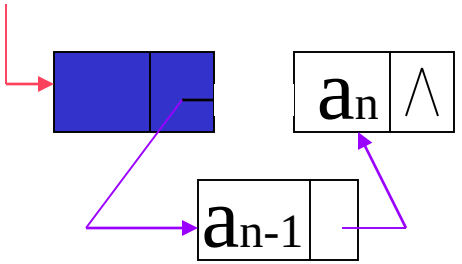
一、建立一个“空表”；



二、输入数据元素a_n，建立结点并插入；



三、输入数据元素a_{n-1}，建立结点并插入；



四、依次类推，直至输入a_1为止。

```
void CreateList_L(LinkList &L, int n) {
  // 逆序输入 n 个数据元素，建立带头结点的单链表
  L = (LinkList) malloc (sizeof (LNode));
  L->next = NULL;     // 先建立一个带头结点的单链表
  for (i = n; i > 0; --i) {
    p = (LinkList) malloc (sizeof (LNode));
    scanf(&p->data);    // 输入元素值
    p->next = L->next; L->next = p;  // 插入
  }
} // CreateList_L
```

算法的时间复杂度为：O(Listlength(L))

回顾 2.1 节中两个例子的算法，看一下当线性表分别以顺序存储结构和链表存储结构实现时，它们的时间复杂度为多少？

例2-1

```
void union(List &La, List Lb) {
   La_len = ListLength(La);     Lb_len =ListLength(Lb);
   for (i = 1;  i <= Lb_len;  i++) {
      GetElem(Lb, i, e);
      if (!LocateElem(La, e, equal( ))
         ListInsert(La, ++La_len, e);
   }//for
} // union
```

算法时间复杂度

控制结构：for 循环

基本操作：GetElem, LocateElem 和 ListInsert

当以链式映像实现抽象数据类型线性表时为：

$O(\text{ListLength}(La) \times \text{ListLength}(Lb))$

例2-2

```
void MergeList(List La, List Lb, List &Lc) {
 InitList(Lc);     i = j = 1;    k = 0;
 La_len = ListLength(La);  Lb_len = ListLength(Lb);
 while ((i <= La_len) && (j <= Lb_len)) {
    GetElem(La, i, ai);  GetElem(Lb, j, bj);
    if (ai <= bj) {
       ListInsert(Lc, ++k, ai);  ++i; }
    else { ListInsert(Lc, ++k, bj);  ++j; }
 }     … …
```

分析比较书P31 算法2.12，书P26算法2.7和书P21 算法2.2

静态链表

静态链表的概念

用一维数组来实现线性链表，这种用一维数组表示的线性链表，称为静态链表。

静态特点：体现在表的容量是一定的。（数组的大小）

链表特点：插入与删除同前面所述的动态链表方法相同

静态链表的类型定义

```
#define  MAXSIZE   1000   // 链表的最大长度
struct Component {
ElemType   data;
int              cur;
} ;
 Component VList[MAXSIZE];
```

游标 cursor

SLinkList类型的数组变量是结构数组，每一数组分量包括两个域：

data：用于存储线性表元素；

cur： 用于存储直接后继元素在数组中的位置

静态链表图示

h=5

0	a_4	-1
1		
2	a_3	0
3		
4		
5	a_1	8
6		
7		
8	a_2	2
9		
10		

数组下标

线性链表图示

h=1020

1010	a_4	0
1012		
1014	a_3	1010
1016		
1018		
1020	a_1	1026
1022		
1024		
1026	a_2	1014
1028		
1030		

地址

静态链表与线性链表的区别？

静态链表的操作

例： 设插入和删除只在表的头上进行

h=7（5，7，2，3）

0		
1	7	9
2		
3	3	-1
4		
5		
6		
7	5	1
8		
9	2	3
10		

表中加入x

- 在VList中找空位置i (比如0)
- VList[i].data=x;
- VList[i].cur=h;
- h=i;

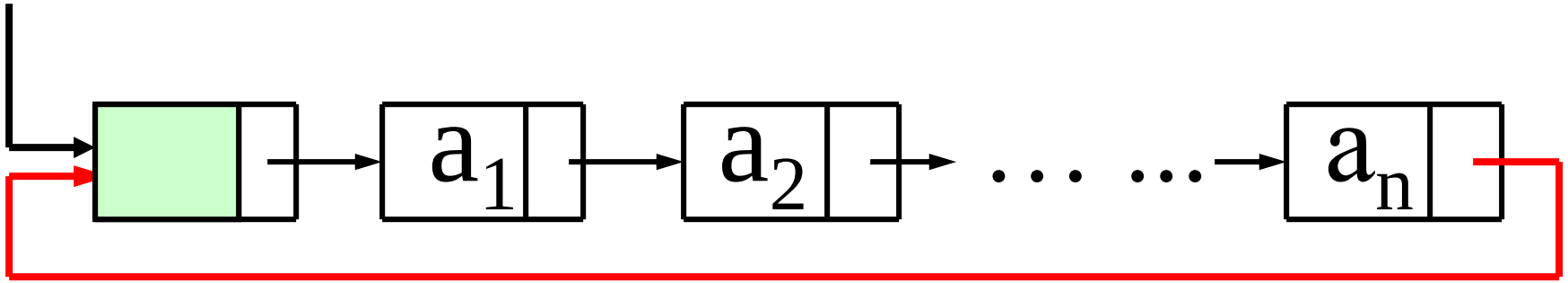
h=0

0	x	7
1	7	9
2		
3	3	-1
4		
5		
6		
7	5	1
8		
9	2	3
10		

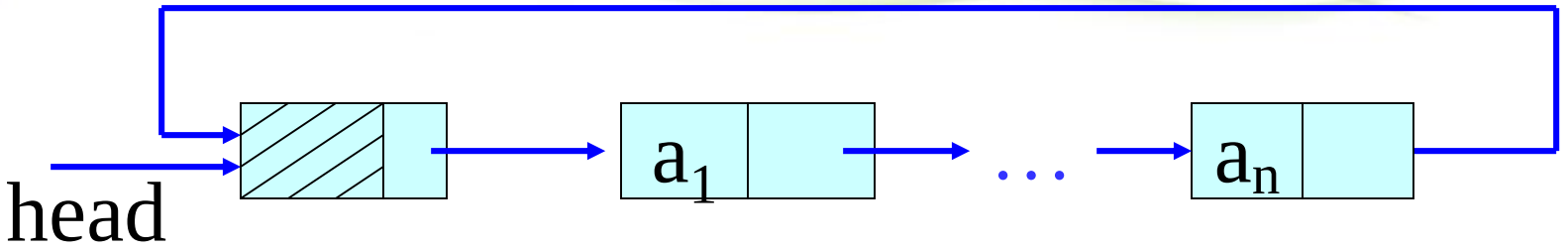
（x，5，7，2，3）

2.3.2 循环链表

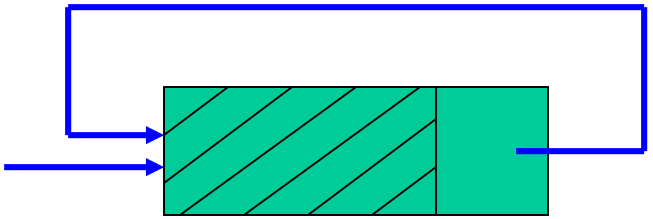
最后一个结点的指针域的指针又指回第一个结点的链表



和单链表的差别仅在于，判别链表中最后一个结点的条件不再是“后继是否为空”，而是“后继是否为头结点”



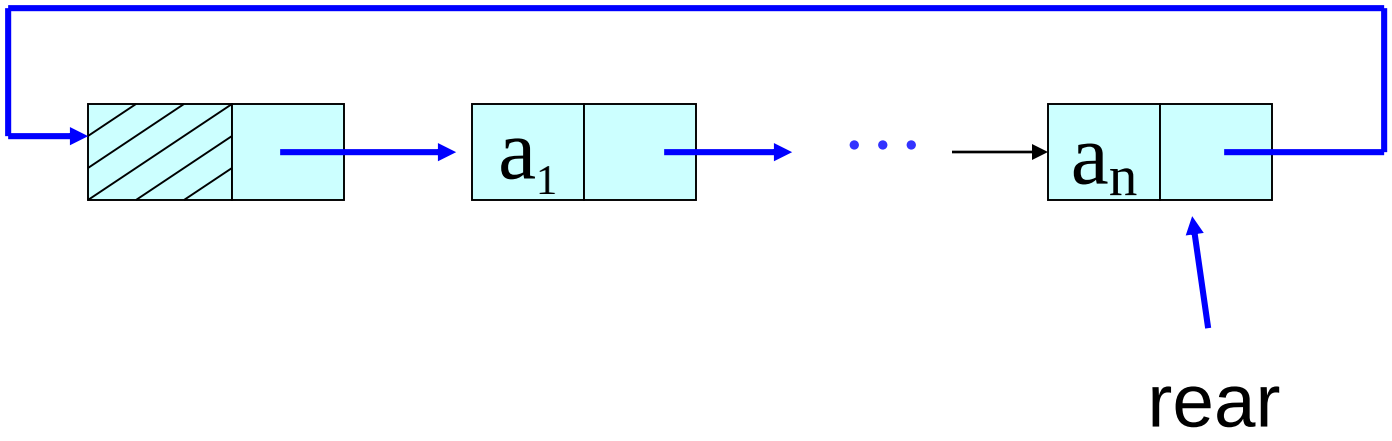
(1) 非空表



(2) 空表

在用头指针表示的单链表中，找开始结点a_1的时间是O(1)，然而要找到终端结点a_n，则需从头指针开始遍历整个链表，其时间是O(n)

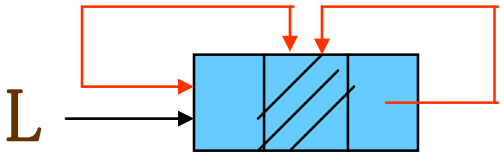
在很多实际问题中，表的操作常常是在表的尾位置上进行，此时头指针表示的单循环链表就显得不够方便.如果改用尾指针rear来表示单循环链表，则查找开始结点a_1和终端结点a_n都很方便，它们的存储位置分别是rear—>next—>next和rear，显然，查找时间都是O(1)。因此，实际中也常采用尾指针表示单循环链表.。



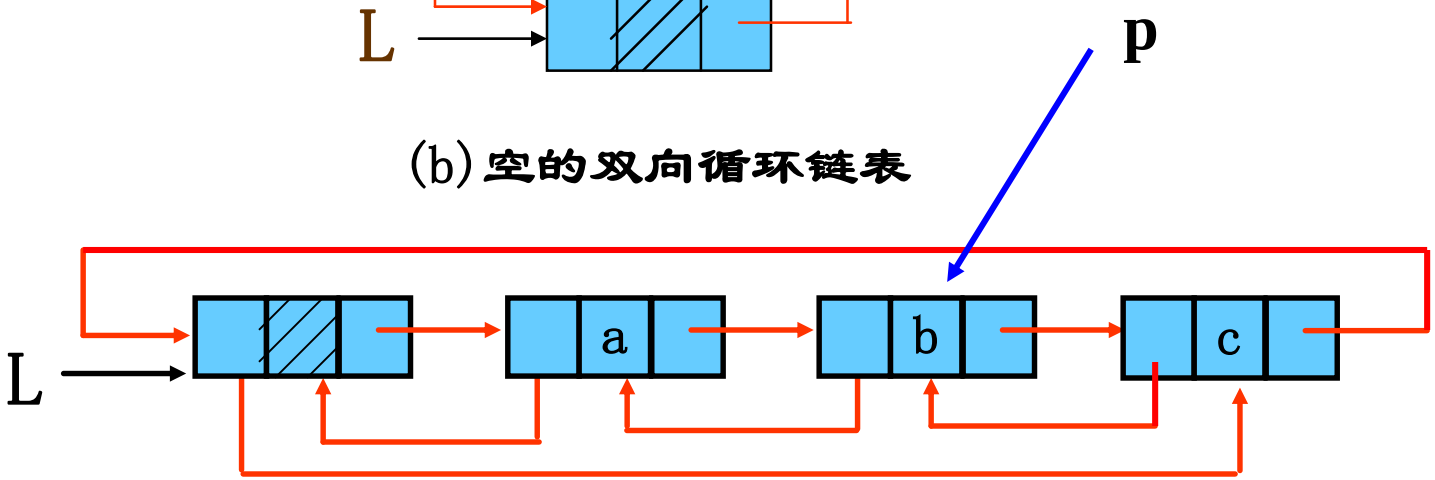
2.3.3 双向链表

```
typedef struct  DuLNode {
    ElemType        data;    // 数据域
    struct DuLNode   *prior;
                             // 指向前驱的指针域
    struct DuLNode  *next;
                             // 指向后继的指针域
} DuLNode, *DuLinkList;
```





(b) 空的双向循环链表



（c）非空的双向循环链表

双链表一般由头指针唯一确定的，将头结点和尾结点链接起来构成循环链表，并称之为双向链表。

设指针p指向某一结点，则双向链表结构的对称性可用下式描述：

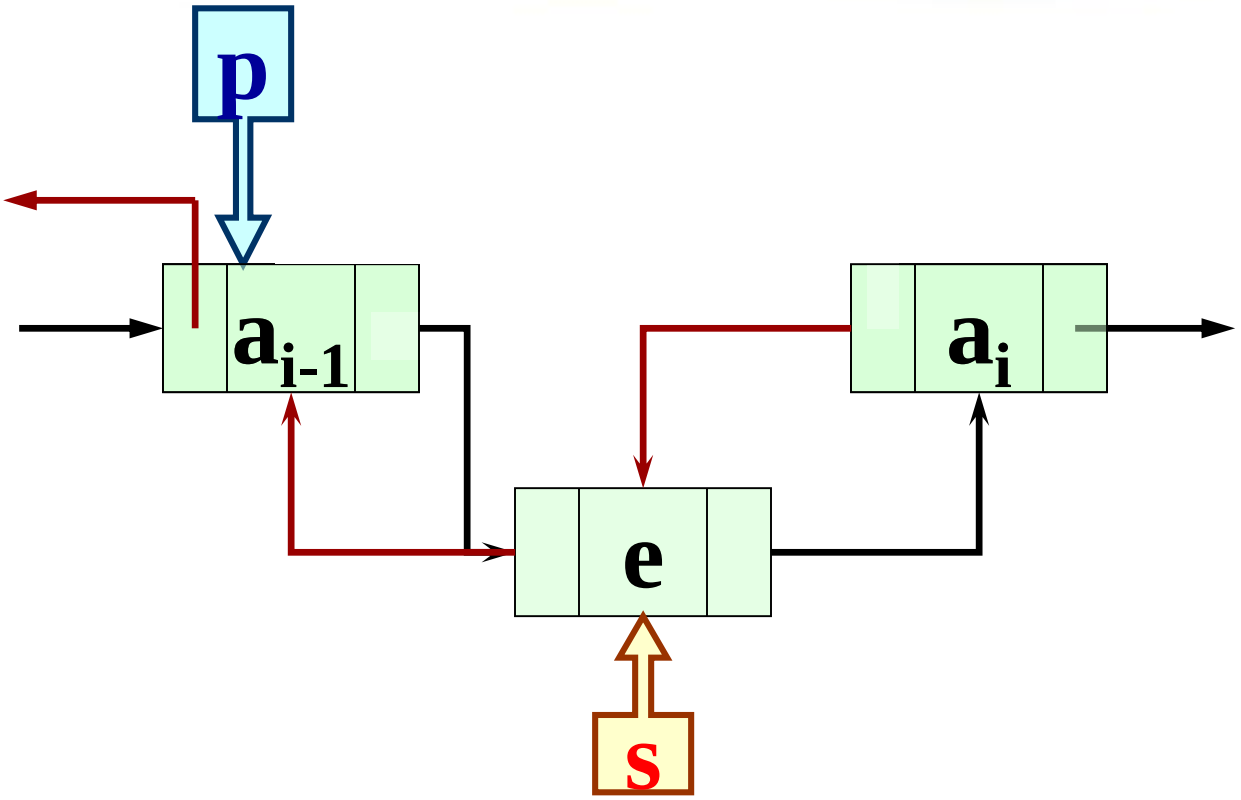
p—>prior—>next=p=p—>next—>prior

双向链表的操作特点：

“查询” 和单链表相同。

“插入” 和“删除”时需要同时修改两个方向上的指针。

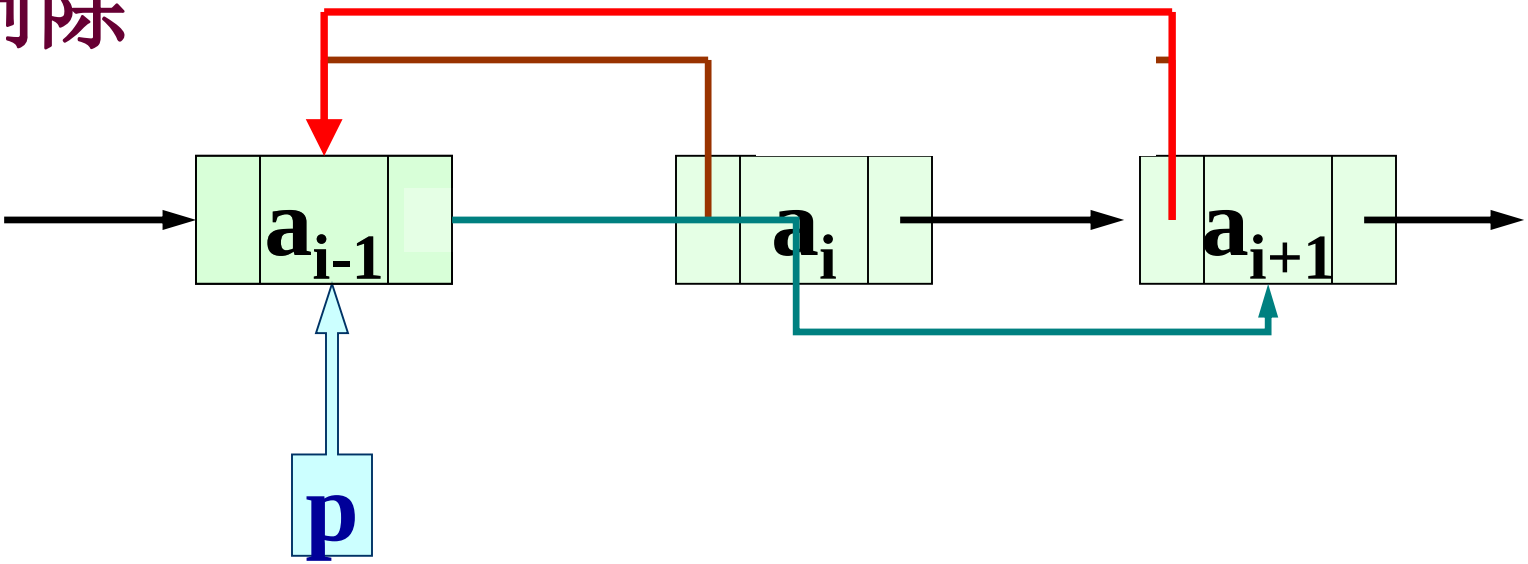
插入



s->next = p->next;　p->next = s;

s->next->prior = s;　s->prior = p;

删除



p->next = p->next->next;

p->next->prior = p;

上机作业 2.3

- 2.3.1 试写出逆转线性单链表的算法。（用C++语言）

- 2.3.2 已知线性表中的元素非递减有序排列，并以带头结点的单链表作存储结构。 试写一高效的算法，删除表中所有值相同的多余元素（即使得操作后的线性表中的所有元素的值均不相同），同时释放被删结点空间，并分析算法的时间复杂度。（用C++语言）

第二章 线性表

- 2.4 一元多项式的表示及相加

一元多项式的表示

一元多项式

$$p_n(x) = p_0 + p_1 x + p_2 x^2 + \ldots + p_n x^n$$

在计算机中，可以用一个线性表来表示:

$P = (p_0, p_1, \cdots, p_n)$

但是对于形如

$$S(x) = 1 + 3x^{10000} - 2x^{20000}$$

但的多项式，上述表示方法是否合适?

一元稀疏多项式

一般情况下的**一元稀疏多项式**可写成

$$P_n(x) = p_1x^{e1} + p_2x^{e2} + \cdots + p_mx^{em}$$

其中： p_i 是指数为 e_i 的项的非零系数，

$$0 \le e_1 < e_2 < \cdots < e_m = n$$

可以下列线性表表示：

$$((p_1, e_1), (p_2, e_2), \cdots, (p_m, e_m))$$

例如：

$$P_{999}(x) = 7x^3 - 2x^{12} - 8x^{999}$$

可用线性表

$$((7, 3), (-2, 12), (-8, 999))$$

表示

抽象数据类型一元多项式的定义

ADT Polynomial {

数据对象：

D＝{ $a_i \mid a_i \in$ TermSet, i=1,2,...,m, m≥0

TermSet 中的**每个元素包含一个表示系数的实数和表示指数的整数 }**

数据关系：

R1＝{ $<a_{i-1}, a_i> \mid a_{i-1}, a_i \in D$, i=2,...,n

且a_{i-1}**中的指数值＜**a_i**中的指数值 }**

基本操作：

CreatPolyn (&P, m)

操作结果：输入 m 项的系数和指数，建立一元多项式 P。

DestroyPolyn (&P)

初始条件：一元多项式 P 已存在。

操作结果：销毁一元多项式 P。

PrintPolyn (&P)

初始条件：一元多项式 P 已存在。

操作结果：打印输出一元多项式 P。

PolynLength(P)

初始条件：一元多项式 P 已存在。

操作结果：返回一元多项式 P 中的项数。

AddPolyn (&Pa, &Pb)

初始条件：一元多项式 Pa 和 Pb 已存在。

操作结果：完成多项式相加运算，即：

Pa = Pa + Pb，并销毁一元多项式 Pb。

SubtractPolyn (&Pa, &Pb)

… …

} ADT Polynomial

存储结构

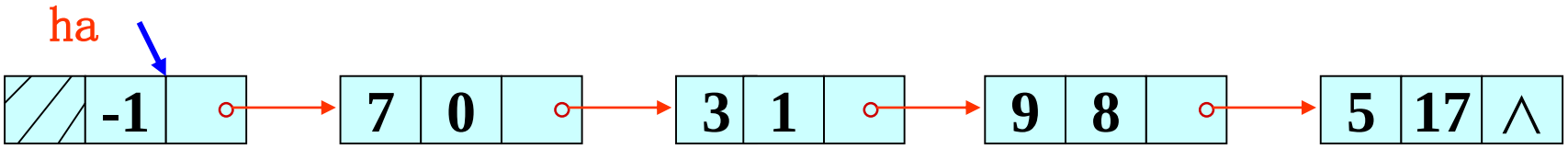
存储结构：用线性链表表示。

coef	expn	next

coef:系数, expn:指数, next:指针

其中，头结点的expn为-1。

例如：多项式 $A(x) = 7 + 3x + 9x^{8} + 5x^{17}$



一元多项式的实现

```
typedef  LinkList   polynomial;
```

// **用带表头结点的有序链表表示多项式**

结点的数据元素类型定义为：

```
typedef struct {      // 项的表示
    float  coef;      // 系数
    int    expn;      // 指数
} term, ElemType;
```

一个带头结点的线性链表类型

```
typedef struct LNode {  // 结点类型
   ElemType       data;
   struct LNode   *next;
} *Link, *Position;

typedef struct {  // 链表类型
   Link  head, tail;     // 分别指向头结点和
                         // 最后一个结点的指针
   int   len;            // 指示链表长度
} LinkList;                                  书p37-38
```

带头结点的线性链表的基本操作

书p37-38

修改两个操作：

Status InsFirst (LinkList **&**L, Link h, Link s);

// 已知h指向线性表L中的一个结点，将s所指的结点插入在h所指的结点之后

Status DelFirst (LinkList **&**L, Link h, Link &q);

//已知h指向线性表L中的一个结点,删除h所指的结点之后的结点并以q返回

有序线性链表

有序链表的基本操作与线性链表有两处不同：

Compare是一个有序判定函数

一、**LocateElem(L, e, &q,**

int(*compare)(ElemType,ElemType))

初始条件：有序表L已存在。

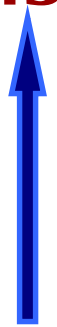
操作结果：若有序表L中存在元素e，则 q指示L中**第一个值为 e 的元素**的位置，并返回函数值TRUE；否则 q 指示**第一个大于 e 的元素的前驱**的位置，并返回函数值FALSE。

例如:

(12, 23, 34, 45, 56, 67, 78, 89, 98, 45)

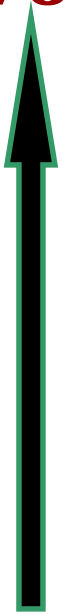
若 e = 45,

则 q 指向



若 e = 88,

则 q 指向



表示值为 88 的元素应插入在该指针所指结点之后。

二、**OrderInsert(&L, e,**

int(*compare)(ElemType,ElemType))

初始条件：有序表L已存在。

操作结果：按有序判定函数 compare()的约定，将值为e的节点插入到有序链表L的适当位置上。

typedef LinkList polynomial;

// **用带表头结点的有序链表表示多项式**

```
Void CreatPolyn ( polynomail &P, int m ) {
  // 输入m项的系数和指数，建立表示一元多项式的有序链表P
    InitList (P); h = GetHead (P);
    e.coef = 0.0;  e.expn = -1;
    SetCurElem (h, e);  // 设置头结点的数据元素
    for ( i=1; i<=m; ++i ) {  // 依次输入 m 个非零项
        scanf (e.coef, e.expn);
        if (!LocateElem ( P, e, q, (*cmp)()) )
            if ( MakeNode (s, e))  InsFirst ( P, q, s ) ;
    }
} // CreatPolyn
```

注意: 1.输入次序不限;

2.指数相同的项只能输入一次。

一元多项式的相加算法

例：求两多项式的和多项式

$$A(x) = 7 + 3x + 9x^{8} + 5x^{17}$$

$$B(x) = 8x + 22x^{7} - 9x^{8}$$

一元多项式相加运算规则：指数相同的项系数相加

A (x)与 B (x)相加的和多项式为

$$C(x) = A(x) + B(x) = 7 + 11x + 22x^{7} + 5x^{17}$$

设多项式A (x) ， B (x)分别用带表头结点的线性链表Pa, Pb表示, 完成：**Pa=Pa+Pb**

一元多项式加法算法主要步骤

分别对两个链表Pa 、Pb进行扫描，设工作指针pa 、 pb分别指向两个多项式当前进行比较的结点，ha指针指向pa的前驱，初始：

ha=Gethead(Pa); pa=ha->next; pb=hb->next;

若pa, pb都不为空：则比较两者指数：

pa->expn < pb->expn： ha,pa后移；

pa->expn = pb->expn：将pb所指结点的系数“加”到pa所指结点的系数上；若和为0，则pa所指结点删除. ha , pa, pb调整

pa->expn > pb->expn : 从表Pb中复制pb所指结点的coef,expn，并将其插入到Pa表pa所指结点之前；

若pb不空：Pb表中从pb开始的结点插入到Pa表尾上

一元多项式加法算法

- 书p43 算法2.22
  - 该算法结果将链表Pb销毁了
  - DelFirst 和 InsFirst的修改

- 上机实验题：
  - 编写相应的C++算法实现一元多项式加法（附加题：一元多项式乘法）

本章小结

- 了解线性表的逻辑结构特性是数据元素之间存在着线性关系，在计算机中表示这种关系的两类不同的存储结构是顺序存储结构和链式存储结构。用前者表示的线性表简称为顺序表，用后者表示的线性表简称为链表。
- 熟练掌握这两类存储结构的描述方法，以及线性表的各种基本操作的实现。
- 能够从时间和空间复杂度的角度综合比较线性表两种存储结构的不同特点及其适用场合。